Diffusion from First Principles

Jai Sharma

6 October 2025

Contents

1 Introduction **2**

2 Forward SDE **4**

3 Fokker-Planck Equation **5**
3.1 Fokker-Planck for Diffusion using Random Walks 5
3.2 Fokker-Planck for Drift 6
3.3 Final Fokker-Planck Equation 7

4 Reverse SDE **7**

5 Boltzmann Distribution **9**
5.1 Method of Lagrange Multipliers 10

6 Einstein Stokes Relation **12**

7 Langevin Sampling **13**
7.1 Drift Term 13
7.2 Diffusion Term 14

8 Fluctuation-Dissipation Theorem **14**

9 Predictor-Corrector Sampling **18**

1 Introduction

The purpose of this text is to introduce the idea of diffusion in Machine Learning from first principles. Diffusion is a physical term, describing the stochastic nature of events like gas molecules dissipating into the surrounding atmosphere or a speck of dust moving through water. The classic implementation of diffusion in Machine Learning is that for image generation, so some of the examples here will allude to that.

Suppose we wish to use diffusion to create an image generation system. As opposed to generating the image directly, say from a neural network model, diffusion views the process of image generation as an iterative procedure. At inference time, we would start from a "random image," or an image with pixels all generated randomly, and use a neural network to determine how the image should be changed to make it more like a "real image." Loosely, we can think of a random image as a real image layered with a lot of noise, and this process can be seen as having a neural network predict the amount of noise to remove from the image at each step of the generation process. Theoretically, over many steps of removing noise, a random image can be transformed into a real image with no noise.

But how are we supposed to get signal from noise? You may have heard of adding noise to an image to make it more blurry or obfuscate it, but removing noise? To train some neural network to achieve this task, diffusion takes the following approach:

1. Collect some examples of real images in a dataset.
2. Simulate the process of adding random noise to a real image till it is completely unrecognizable (e.g. it is effectively a random image).
3. Train a neural network to predict the noise-added at each step.
4. At inference time, generate a random image. Use the neural network to determine what noise to remove, or how to update the image, at each timestep. Repeat this process of removing noise from the image for many steps until the image is a real image.

We can visualize each image as a point in high dimensional space. For example, each image in the MNIST dataset of hand drawn images has shape $(28, 28)$ and $28 \times 28 = 784$ pixels. Therefore, each image can be thought of as a vector of length 784, or a point in 784-dimensional space.

If we generate each pixel of a random image from the normal distribution $N(0, 1)$, we might expect the distribution of random images to be centered at the origin. The distribution of real images might not be centered at the origin, and there might be multiple modalities of real images (e.g. if we had images of people's faces and cars in the same dataset, we might expect the distribution of images to have one mode corresponding to each type of image). The variance of the distribution of real images might be very small as well, since it is probably very unlikely to generate a realistic image by picking each pixel at random. In other words, the region of real images is probably very small.

The idea that our dataset might have multiple modes (e.g. people's faces and cars) is a powerful one, as it makes us believe that the process of generating images need not be deterministic given our random starting point. Even then, the idea of reversing the noising process implies that there should be some deterministic component in how we are updating our image point. As an analogy, if our image was a speck of dust floating in water, we might expect a combination of a random force from collisions with the surrounding water molecules and a deterministic force corresponding to the water current or the force due to gravity. Specifically, there are two types of forces acting on the speck of dust:

- Random forces due to the constant bombardment of water particles moving in random directions. Call this the *diffusion* force.
- Deterministic forces due to the nature of the water current, gravity, or some external force. Call this the *drift* force.

Suppose that the water has no deterministic current, and the only external force acting on the speck of dust is gravity. Let $P(x, t)$ be the probability that the speck of dust is at position x at time t. At time $t = 0$, the speck of dust is at its starting location x_0. We would like to know what is the most likely location we can find the speck of dust at some time t in the future, where likelihood is determined by the function $P(x, t)$. Intuitively, perhaps we expect the speck of dust to be closer to the bottom of the water container since that corresponds to a lower gravitational potential energy.

However, the speck's movement through the water is non-deterministic, so it might move randomly in the xy plane. Also, its movemoment might be inhibited by a drag force opposing its motion. So, we cannot know the exact position. Instead, we could choose to simulate the speck's journey many times through the liquid, and then find regions with high density at time t. This would be a way of empirically sampling the distribution of the position of the speck at time t. In that way, we might be able to surmise some information about the true probability function $P(x, t)$.

Instead of sampling individual trajectories, a more helpful model might be to create a vector field of each point x in the container. This vector field would tell us the force (or acceleration) exerted on the particle on average at each position x at time t.

Another useful model to gain a better understanding of $P(x)$ might be to think about $\frac{dP(x,t)}{dt}$, or how the probability density of each point in the container changes with respect to time. Over time, we might observe that certain locations of the tank tend to be probability sources, where probability flows out of them on average and into surrounding areas. Other locations may tend to be probability sinks, where probability flows into them on average from surrounding areas. A physical analogy of this model is to imagine a cloud of smoke in the container. At each time t, the cloud of smoke is more dense where the speck of dust is more likely to be, and less dense where the speck is likely not. Measuring $\frac{dP(x,t)}{dt}$ is like measuring the rate of change of the density of the smoke at location x at time t.

Loosely, the rate of change of the density at location x at time t, $\frac{dP(x,t)}{dt}$, might be dependent on the current density and the deterministic force (or acceleration). The amount of probability flowing from a location into surrounding areas should be proportional to both the location's density and expected acceleration. (This is similar to Newton's second law $F = ma$.) Therefore, our first and second models are related at some fundamental level.

Going back to the context of images, we can think of our random image as the starting point for the speck of dust in the water. The problem of finding the location of the speck of dust at some time t in the future is equivalent to finding the image when denoised after t timesteps. We can roughly move the image closer to the distribution of real images while adding some noise during the generation process to make sure that the result is non-deterministic and that we do not get stuck in some local maxima.

In this context, x corresponds to a possible image in the image space, and t represents the timestep of denoising. Then, suppose $P(x)$ is the probability that an image is real. We wish to maximize $P(x)$ for the image x we are currently generating. It is actually computationally easier to maximize $\log P(x)$ instead; these objectives are also equivalent since log is a monotonically increasing function. (Note that we will use log to denote the logarithm with base e, not 10, unless explicitly stated.)

Suppose our current noisy image is x_0. How should we update our image to marginally increase $\log P(x_0)$? We can take the gradient with respect to the current location x_0! The vector $\nabla_{x_0} \log P(x_0)$ represents the most immediate direction of probability ascent. By moving a small amount in this direction, we increase the probability that our image is real. The update to find the next image x_1 would be:

$$x_1 = x_0 + \epsilon \cdot \nabla_{x_0} \log P(x_0)$$

for some small update stepsize $\epsilon > 0$. Theoretically, x_1 is marginally more likely to be a real image compared to x_0. Then, we could just repeat the process for x_1, finding $\nabla_{x_1} \log P(x_1)$ and updating to get x_2:

$$x_2 = x_1 + \epsilon \cdot \nabla_{x_1} \log P(x_1).$$

Continuing in this way, this process would be the opposite of gradient descent, the algorithm used for learning the parameters of a neural network, since we are doing a *maximization* of probability instead of a *minimization* of loss.

We can view this as our current image x being a point in high dimensional space, and its movement is determined by $\nabla_x \log P(x)$, the gradient of the log probability at that point. This means that the vector field that would be important to learn, say for a neural network, is $x \to \nabla_x \log P(x)$. Then, we would be able to start at some point and just move according to the learned vector field.

Note that our current update function is deterministic, so the same starting point will arrive at the same image after a fixed number of timesteps. It is also possible that multiple starting points (or a region of starting points) will arrive at the same image after a fixed number of timesteps. This would be undesirable because ideally, we would like to generate many types of images. This is a good reason to add some kind of randomness to our update, similar to the random forces experienced by the speck of dust in the physical example.

Over the next few sections, we will unpack these concepts and give more insight into some of the intuitions discussed here.

2 Forward SDE

In the example of the speck of dust in a container of water, there are two main types of forces acting on the speck:

1. Drift: deterministic forces acting on the particle.
2. Diffusion: random forces acting on the particle.

In the context of image generation, we wish to learn how to reverse the noising process. The first step is to simulate the forward process of noising real images, and then train a neural network to somehow learn to undo this noising process. The following is the main equation for the forward noising process Stochastic Differential Equation (SDE):

$$dx = \underbrace{f(x,t)\,dt}_{\text{drift}} + \underbrace{g(t)\,dw}_{\text{diffusion}}.$$

At inference time, we would like to be able to start with a random image, where each pixel is generated from $N(0,1)$, and generate real images. So, we need to make sure that dx is moving the real image x to this distribution eventually. The $f(x,t)\,dt$ deterministic drift term, will move x closer to 0 to ensure the mean converges to 0. The $g(t)\,dw$ random diffusion term will constantly add random noise to eventually guarantee that x has a variance of 1.

The amount of noise we add is dependent on the timestep. Specifically, as time goes on, we tend to add more noise. We can create a noise scheduler $\beta(t)$ as the following:

$$\beta(t) = \beta_{\min} + t(\beta_{\max} - \beta_{\min}); \beta_{\min} = 0.0001, \beta_{\max} = 0.02, t \in [0,1].$$

As in the above example, the noise scheduler $\beta(t)$ is often linear in t, and is capped at some minimum and maximum (both greater than 0, but not necessarily the same magnitude). Then, an example of $f(x,t)$ and $g(t)$ might be:

$$f(x,t) = -\frac{1}{2}x\beta(t)$$
$$g(t) = \sqrt{\beta(t)}.$$

We do not want any kind of deterministic component in the diffusion term, so we can enforce the condition that $\mathbb{E}[dw] = 0$. So, to find $\mathbb{E}[x]$, solve the forward SDE while ignoring the term for diffusion. Let $m(t)$

denote the mean of x at time t:

$$\begin{aligned}
dm &= f(m,t)\,dt \\
dm &= -\frac{1}{2}m\beta(t)\,dt \\
\frac{dm}{m} &= -\frac{1}{2}\beta(t)\,dt \\
\int \frac{dm}{m} &= \int -\frac{1}{2}\beta(t)\,dt \\
ln(m) &= \int -\frac{1}{2}\beta(t)\,dt + C \\
m &= e^{\int -\frac{1}{2}\beta(t)\,dt + C} = e^C \cdot e^{\int -\frac{1}{2}\beta(t)\,dt} \\
m(t) &= m(0) \cdot e^{\int_0^t -\frac{1}{2}\beta(t)\,dt}
\end{aligned}$$

Notice how if $\beta(t) > 0$, then $m(t)$ is an exponentially dampened version of the original mean $m(0)$. Therefore, $m(t)$ will eventually reach the origin. Also, we can define $dw = \sqrt{dt} \cdot z$, where $z \sim N(0,1)$ is the source of the randomness. Loosely, the square root in the $\sqrt{dt}$ and $\sqrt{\beta(t)}$ terms is due to the following idea: Suppose we take a random "drunkard's walk" on a 1D line, starting from the origin ($x = 0$) and taking a step in a random direction (left 1 unit or right one unit, each with $\frac{1}{2}$ probability) at every timestep. The variance of each step is

$$\left[\frac{1}{2}1^2 + \frac{1}{2}(-1)^2\right] - \left[\frac{1}{2}1 + \frac{1}{2}(-1)\right]^2 = 1.$$

Therefore, the variance of x after 100 timesteps is $100 \cdot 1 = 100$, so the standard deviation is $\sqrt{100} = 10$ units. This means that after 100 timesteps, we expect to be about 10 steps away from the origin. Therefore, the square root function can be seen as the normalization function for the noise applied in the diffusion term.

In summary, this forward SDE describes the noising process of stochastic systems, whether it be a speck of dust moving in a water container or an image moving from real image space to complete noise. The two main components of the forward SDE are the drift term, which is deterministic, and the diffusion term, which is random.

3 Fokker-Planck Equation

In this section, we will come up with a model for $\frac{dP(x,t)}{dt}$, or the change in probability at each location with respect to time, for a stochastic system under the influence of a differential equation described in the previous section. The important analogy here is to think of probability as a cloud of smoke. Regions with dense smoke correspond to a higher probability that we might find our speck of dust or image point at that location at time t, and regions with less dense smoke correspond to a lower probability. We wish to analyze the flow of this smoke. Certain regions are likely *sources* of probability, such as the region of random images, while others are *sinks* of probability, such as the region of real images. We handle models for $\frac{dP(x,t)}{dt}$ separately for drift and diffusion.

3.1 Fokker-Planck for Diffusion using Random Walks

Diffusion is the random process of noise being added to our system, such as through random collisions between our speck of dust and surrounding water molecules. Let us first examine a simpler random process: taking a random walk on a 1D number line.

Suppose we start at the origin $x = 0$, and we run an experiment where we walk either left 1 unit or right 1

unit at random. Denote $P(x,t)$ as the probability that we are at position x at time t. The probability that we are at position x at time $t+1$, $P(x,t+1)$ can be expressed in the following way:

$$P(x,t+1) = \frac{1}{2}P(x-1,t) + \frac{1}{2}P(x+1,t).$$

In order to be at position x at time $t+1$, we must be at either $x-1$ at time t and take a step to the right (with probability $\frac{1}{2}$) or be at $x+1$ at time t and take a step to the left (also with probability $\frac{1}{2}$).

This is the alternative in the discrete case, where our stepsize is 1 unit at each timestep. For the continuous case, we can just change the increments for x and t to dx and dt, respectively:

$$P(x,t+dt) = \frac{1}{2}P(x-dx,t) + \frac{1}{2}P(x+dx,t).$$

We can take the Taylor Series expansion for each of the three terms:

$$\begin{aligned}
P(x,t+dt) &= P(x,t) + \frac{\partial P}{\partial t}dt + \frac{1}{2}\frac{\partial^2 P}{\partial t^2}dt^2 + \cdots \\
P(x-dx,t) &= P(x,t) - \frac{\partial P}{\partial x}dx + \frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2 + \cdots \\
P(x+dx,t) &= P(x,t) + \frac{\partial P}{\partial x}dx + \frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2 + \cdots
\end{aligned}$$

For the first term, we truncate at the first order, and for the other two, we truncate at the second order. Then, we substitute back into the original equation:

$$\begin{aligned}
P(x,t+dt) &= \frac{1}{2}P(x-dx,t) + \frac{1}{2}P(x+dx,t) \\
P(x,t) + \frac{\partial P}{\partial t}dt &= \frac{1}{2}\left[P(x,t) - \frac{\partial P}{\partial x}dx + \frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2\right] + \frac{1}{2}\left[P(x,t) + \frac{\partial P}{\partial x}dx + \frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2\right] \\
P(x,t) + \frac{\partial P}{\partial t}dt &= P(x,t) + \frac{1}{2}\left[\frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2\right] + \frac{1}{2}\left[\frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2\right] \\
\frac{\partial P}{\partial t}dt &= \frac{1}{2}\frac{\partial^2 P}{\partial x^2}dx^2 \\
\frac{\partial P}{\partial t} &= \frac{dx^2}{2dt}\frac{\partial^2 P}{\partial x^2}
\end{aligned}$$

If we define $D = \frac{dx^2}{2dt}$, then we get:

$$\frac{\partial P(x,t)}{\partial t}_{\text{diffusion}} = D\frac{\partial^2 P(x,t)}{\partial x^2}.$$

Therefore, the change in probability with respect to time is proportional to the second derivative of the probability with respect to the location.

3.2 Fokker-Planck for Drift

The deterministic force acting on the smoke cloud of probability is dependent on the current density of probability at each location as well as the direction of the vector field of deterministic acceleration. We can view $f(x,t)$ as this vector field, as this represents the average direction of the vector field of acceleration on the speck of dust. Additionally, the current density at a location is given by $P(x,t)$ itself. So, the total amount of probability flow, also known as the flux, is given by $f(x,t) \cdot P(x,t)$. We can find the net outflow

of flux from a point in space by taking the divergence of this: $\nabla_x[f(x,t) \cdot P(x,t)]$. To find the net inflow of flux, we can take the negative of this:

$$\frac{\partial P(x,t)}{\partial t}_{\text{drift}} = -\nabla_x[f(x,t) \cdot P(x,t)].$$

The net inflow of flux is equivalent to the net inflow of probability to a point. Consider the following analogy: Suppose we store colored gas (i.e. the gas is red smoke, so we can see it) in a small bottle. Then, we break the bottle in the middle of a room, and the gas dissipates. Suppose for sake of argument that the gas is released from a single point in space. Due to deterministic dispersion forces such as air draft, gravity, or air buoyancy, there might be a net outflow of probability from the original point the gas was released. Perhaps the gas is less dense, so the gas rises. Then, there might be a net inflow of probability to a point just below the roof of the room.

3.3 Final Fokker-Planck Equation

Now, we can put both pieces together to get the final Fokker-Planck equation:

$$\frac{\partial P(x,t)}{\partial t} = \frac{\partial P(x,t)}{\partial t}_{\text{drift}} + \frac{\partial P(x,t)}{\partial t}_{\text{diffusion}} = \underbrace{-\nabla_x[f(x,t) \cdot P(x,t)]}_{\text{drift}} + \underbrace{D\frac{\partial^2 P(x,t)}{\partial x^2}}_{\text{diffusion}}.$$

As previously mentioned, this is a good model for how the cloud of probability changes over time. Rather than sampling individual trajectories of the speck of dust, we can track the entire distribution of all potential trajectories. Additionally, even though each individual sample might be non-deterministic, the change in the probability distribution of the particle is deterministic, as evidenced by the above equation. This allows us to understand the probability distribution at a location at some point in time without having to sample many trajectories.

4 Reverse SDE

Previously, we have discussed how image generation using diffusion requires first applying noise to images over many timesteps until the image is unrecognizable. Then, we somehow train a neural network to "go backwards" and turn a random image into a real image. This section will cover the methodology behind reversing the noising process.

We view the diffusion noising process as a stochastic process, so we take the same approach with the denoising process. We want a deterministic component, or a "reverse drift" term, and a random component, or a diffusion term. Let us work on finding the reverse drift term. Here are the Fokker-Planck equations for the forward and reverse processes:

$$\frac{\partial P(x,t)}{\partial t} = -\nabla_x[f(x,t) \cdot P(x,t)] + D\frac{\partial^2 P(x,t)}{\partial x^2} \qquad \text{(forward noising process)}$$

$$\frac{\partial P(x,t)}{\partial(-t)} = -\nabla_x[\overline{f}(x,t) \cdot P(x,t)] + D\frac{\partial^2 P(x,t)}{\partial x^2} \qquad \text{(reverse noising process)}$$

Note that we have denoted the reverse drift term as $\overline{f}(x,t)$ in the reverse noising process. Additionally, note that in the reverse noising process, we take the partial derivative with respect to $-t$ since we are going backwards in time. Therefore, the reverse noising process can be rewritten in the following way:

$$\frac{\partial P(x,t)}{\partial t} = -\left(-\frac{\partial P(x,t)}{\partial t}\right) = -\frac{\partial P(x,t)}{\partial(-t)} = \nabla_x[\overline{f}(x,t) \cdot P(x,t)] - D\frac{\partial^2 P(x,t)}{\partial x^2}.$$

Then, we can set this equal to the forward noising process equation:

$$\begin{aligned}
\frac{\partial P(x,t)}{\partial t} = -\nabla_x[f(x,t)\cdot P(x,t)] + D\frac{\partial^2 P(x,t)}{\partial x^2} &= \nabla_x[\overline{f}(x,t)\cdot P(x,t)] - D\frac{\partial^2 P(x,t)}{\partial x^2} \\
\nabla_x[f(x,t)\cdot P(x,t)] + \nabla_x[\overline{f}(x,t)\cdot P(x,t)] &= 2D\frac{\partial^2 P(x,t)}{\partial x^2} \\
\nabla_x[(f(x,t)+\overline{f}(x,t))\cdot P(x,t)] &= 2D\frac{\partial^2 P(x,t)}{\partial x^2} \\
(f(x,t)+\overline{f}(x,t))\cdot P(x,t) &= 2D\frac{\partial P(x,t)}{\partial x} \qquad \text{(integrate both sides)} \\
f(x,t)+\overline{f}(x,t) &= 2D\frac{1}{P(x,t)}\frac{\partial P(x,t)}{\partial x} \\
\overline{f}(x,t) &= -f(x,t) + 2D\frac{1}{P(x,t)}\frac{\partial P(x,t)}{\partial x}.
\end{aligned}$$

Note that $\frac{d}{dx}\log(u) = \frac{1}{u}\frac{du}{dx}$, so we can perform u-substitution where $u = P(x,t)$:

$$\overline{f}(x,t) = -f(x,t) + 2D\nabla_x \log P(x,t).$$

Let us write the reverse SDE in the same form as the forward SDE, but with $-dt$ instead of dt and $d\overline{w}$ instead of dw (because we are going backwards in time):

$$dx = \overline{f}(x,t)(-dt) + g(t)\, d\overline{w}.$$

Here, $d\overline{w}$ is defined in the same way as dw, as both are *Wiener* processes sampled from the Gaussian normal distribution $N(0, dt)$. Substituting $\overline{f}(x,t)$, we get the final reverse SDE:

$$\begin{aligned}
dx &= [-f(x,t) + 2D\nabla_x \log P(x,t)](-dt) + g(t)\, d\overline{w} \\
dx &= [f(x,t) - 2D\nabla_x \log P(x,t)]dt + g(t)\, d\overline{w}.
\end{aligned}$$

It turns out that we can write $2D$ in terms of $g(t)$:

$$\begin{aligned}
\text{Var}(dx) = \mathbb{E}[(dx)^2] - (\mathbb{E}[dx])^2 &= \mathbb{E}[(dx)^2] \\
&= \mathbb{E}[g(t)^2 (dW_t)^2] = g(t)^2\, \mathbb{E}[(dW_t)^2] = g(t)^2\, dt \\
&= 2D\, dt
\end{aligned}$$

For the second line, note that $g(t)$ is deterministic at time t. Also, $\mathbb{E}[(dW_t)^2] = \text{Var}[dW_t] + (\mathbb{E}[dW_t])^2 = \text{Var}[dW_t] + 0 = dt$. For the third line, note that $\mathbb{E}[x^2] = 2Dt$, so $\mathbb{E}[(dx)^2] = 2D\, dt$. Therefore,

$$2D = g(t)^2.$$

So, we can rewrite the previous result as

$$dx = [f(x,t) - g(t)^2 \nabla_x \log P(x,t)]\, dt + g(t)\, d\overline{w}.$$

Recall that we know $f(x,t)$ and $g(t)$, as these were design choices for the noise scheduling in the forward noising process. The only term here that is intangible is $\nabla_x \log P(x,t)$. As stated earlier, one of the approaches to diffusion in Machine Learning is to predict $\nabla_x \log P(x,t)$ to help us understand the vector field of the reverse noising process.

Call this model $s_\theta(x,t) \approx \nabla_x \log P(x,t)$, where the neural network is parameterized by model weights θ. Then, this becomes:

$$dx = [f(x,t) - g(t)^2 \nabla_x \log P(x,t)]\, dt + g(t)\, d\overline{w} \approx [f(x,t) - g(t)^2 s_\theta(x,t)]\, dt + g(t)\, d\overline{w}.$$

As a short summary, we now have a forward SDE and a reverse SDE. We can use numerical differential equation solvers on the forward SDE to simulate the process of arriving at the noisy version of a real image.

Then, once we learn $s_\theta(x,t)$, we can apply numerical differential equation solvers on the reverse SDE to simulate the process of arriving at the denoised version of a random image. Essentially, sampling this SDE allows us to find one possible path from the region of random images to the region of real images. For example, we would descritize the continuous differential equation to

$$\Delta x = [f(x,t) - g(t)^2 s_\theta(x,t)]\Delta t + g(t)\sqrt{\Delta t}\cdot z,$$

where $dw = \sqrt{dt}\cdot z \approx \sqrt{\Delta t}\cdot z$. Then, we perform the following update to our current random image:

$$x_{t-\Delta t} \approx x_t - \Delta x.$$

Note that we have to subtract Δx because we treated dt as a positive number in our proof, and $-dt$ was the timestep for the reverse process. We repeat this reverse noising process until we arrive at x_0.

5 Boltzmann Distribution

So far, we have been viewing points in space through the lens of probability. While this is a powerful model, there are other intuitions we can borrow from physics. For example, we know that systems tend toward lower potential energy. If I release a ball from the top of a hill, we expect the ball to roll down the hill to a lower height, as the lower height corresponds to lower potential energy. In this section, we will develop the notion of measuring the density of a location as a function of the potential energy of the state.

In order to better formalize our study of energy states, we design the following combinatorial setup: Suppose we have n balls that can be placed into any one of $n+1$ bins, labeled $(0,1,2,\cdots,n)$ corresponding to the bin's energy level. Consider the problem of placing each of the n balls into one of the $n+1$ bins so that the total energy (i.e. the sum of the energy levels of all of the bins the balls are in) is equal to n.

For example, suppose we want to place 3 balls into 4 bins, labeled $(0,1,2,3)$. We call each solution where the balls are identical a macrostate, and each solution where the balls are distinguishable a microstate. Here are the macrostates for this problem:

Macrostate	E_0	E_1	E_2	E_3
A	0	3	0	0
B	1	1	1	0
C	2	0	0	1

In macrostate A, all 3 balls are in bin E_1, so the total energy is $1+1+1=3$, which is valid. In macrostate B, we have one ball in each of E_0, E_1, and E_2, so the total energy is $0+1+2=3$, which is valid. Finally, in macrostate C, we have two balls in E_0 and one ball in E_3, so the total energy is $0+0+3=3$, which is also valid. Therefore, each of the macrostates is valid.

For each macrostate, we can find the number of microstates by simply considering the number of permutations when the balls are distinguishable. For example, in macrostate C, we can decide which of the three balls to keep in E_3. Once we decide that, the other two balls must be kept in E_0. Therefore, the number of microstates for macrostate C is 3, which is also computable by $\frac{3!}{2!\cdot 1!} = 3$.

In general, suppose that $(n_0, n_1, n_2, \cdots, n_n)$ is the number of balls in each of the $n+1$ bins for a particular macrostate. The number of microstates for this macrostate can be computed using the following formula:

$$W = \frac{n!}{n_0!\cdot n_1!\cdot n_2!\cdots n_n!}.$$

We can determine how likely a macrostate is based on the number of corresponding microstates. To find the most probable macrostate, we find the macrostate with the most number of microstates. This corresponds to maximizing W, which is equivalent to maximizing $\log W$.

We bring up $\log W$ because the logarithm of factorials have a very good approximation known as Stirling's approximation:

$$\log(N!) \approx N \log N - N.$$

Then, we can expand and reduce $\log W$ (here, we use N to denote n, the number of balls, for clarity):

$$\begin{aligned} \log W &\approx N \log N - N - \sum_{i=0}^{n} (n_i \log(n_i) - n_i) \\ &= N \log N - \sum_{i=0}^{n} n_i \log(n_i) \qquad \left(\text{since } N = \sum_{i=0}^{n} n_i \right). \end{aligned}$$

We can maximize this expression using the method of Lagrange multipliers. Here are the constraints of the problem:

1. Number of balls is N $\qquad (n_0 + n_1 + n_2 + \cdots + n_j + \cdots + n_n) - N = 0$
2. Total energy is N $\qquad (n_0 E_0 + n_1 E_1 + n_2 E_2 + \cdots + n_j E_j + \cdots + n_n E_n) - N = 0$

5.1 Method of Lagrange Multipliers

This subsection is a quick crash course on Lagrange multipliers. Suppose we wish to optimize some function $f(x, y)$ subject to the constraint $g(x, y) = c$, where c is a constant.

At the optimal point, the gradient of the objective function and constraint point in the same direction. To see why this is intuitively, suppose that the gradient of the objective function and constraint do not point in the same direction. Then, the variables x, y can be changed (either in the direction of the gradient of the constraint function or in the opposite direction of the gradient of the constraint function) to produce a more optimal value for the objective function. Mathematically, this can be expressed as one vector being a multiple of another:

$$\nabla f = \lambda \nabla g$$

or equivalently:

$$\nabla f - \lambda \nabla g = 0.$$

In general, we can optimize a function f under constraints $g_1, g_2, \cdots, g_n$ by writing the following:

$$\nabla f - \lambda_1 \nabla g_1 - \lambda_2 \nabla g_2 - \cdots - \lambda_n \nabla g_n = 0.$$

Note that the gradient is taken with respect to one of the variables we are optimizing for. Going back to our examples of balls and bins, the following is the gradient with respect to some n_j for each of the constraints:

1. Number of balls is N $\qquad \dfrac{\partial}{\partial n_j}[(n_0 + n_1 + n_2 + \cdots + n_j + \cdots + n_n) - N] = 1$
2. Total energy is N $\qquad \dfrac{\partial}{\partial n_j}[(n_0 E_0 + n_1 E_1 + n_2 E_2 + \cdots + n_j E_j + \cdots + n_n E_n) - N] = E_j$

Additionally, here is the gradient of the objective function with respect to n_j (we use the product rule):

$$\frac{\partial}{\partial n_j} \log W = -\log n_j - 1.$$

Applying the method of Lagrange multipliers, we get the following equation:

$$\nabla f - \lambda_1 \nabla g_1 - \lambda_2 \nabla g_2 = -\log n_j - 1 - \lambda_1(1) - \lambda_2 E_j = 0.$$

Therefore,

$$n_j = e^{-(\lambda_1+1)} \cdot e^{-\lambda_2 \ E_j} = Ce^{-\lambda_2 \ E_j},$$

where $C = e^{-(\lambda_1+1)}$ is a constant that depends on N. From this, we conclude the following:

$$\rho(E_j) \propto e^{-\lambda_2 \ E_j}.$$

In other words, the number of balls (and the density of balls) at energy level E_j is proportional to some exponential decay function of E_j. This means that higher energy levels have an exponentially smaller concentration of balls. This aligns with our beliefs that systems tend toward lower potential energy with very high probability. We are almost at the Boltzmann distribution.

Let us do a bit of investigation into λ_2. Suppose that there exist systems A and B that can touch and exchange energy with each other. Then, $W_{\text{total}} = W_A \times W_B$. Our physical intuition tells us that at equilibrium, the temperature of both systems is equal. Otherwise, the systems can exchange energy to reach this equilibrium. For example, hot tea will cool down in a room, since the heat from the tea is released into the environment. This can be written mathematically as $T_A = T_B$. We can take the log of both sides of the previous equation and take the derivative with respect to E_A:

$$\log W_{\text{total}} = \log W_A + \log W_B = 0$$
$$\frac{d}{dE_A} \log W_{\text{total}} = \frac{d}{dE_A}[\log W_A + \log W_B] = \frac{d}{dE_A} \log W_A + \frac{d}{dE_A} \log W_B = 0.$$

We set this equal to 0 to find the maximum value of W_{total}, or the total number of microstates. Since the energy entering system A is the same as the energy leaving system B, we know that $dE_A = -dE_B$. So, the equation can be written in the following way:

$$\frac{d}{dE_A} \log W_A + \frac{d}{-dE_B} \log W_B = \frac{d}{dE_A} \log W_A - \frac{d}{dE_B} \log W_B = 0.$$

In other words,

$$\frac{d(\log W_A)}{dE_A} = \frac{d(\log W_B)}{dE_B}.$$

Before we proceed, note the physics definitions for entropy and temperature:

1. Entropy	$S = k_B \log W$
2. Temperature	$\frac{1}{T} = \frac{dS}{dE}$

Here, k_B is known as the Boltzmann constant. From the definition of entropy, we know that $\log W = \frac{S}{k_B}$.

Now, suppose that we add a tiny amount of energy to the whole system. Here is the change in $\log W$:

$$d(\log W) = \frac{\partial(\log W)}{\partial n_0} dn_0 + \frac{\partial(\log W)}{\partial n_1} dn_1 + \cdots + \frac{\partial(\log W)}{\partial n_n} dn_n.$$

From the method of Lagrange multipliers from before, we know that

$$\frac{\partial(\log W)}{\partial n_j} dn_j = \lambda_1(1) + \lambda_2 E_j.$$

Therefore, the equation becomes

$$d(\log W) = [\lambda_1(1) + \lambda_2 E_0]dn_0 + [\lambda_1(1) + \lambda_2 E_1]dn_1 + \cdots + [\lambda_1(1) + \lambda_2 E_n]dn_n = \lambda_1 \sum_{i=0}^{n} dn_i + \lambda_2 \sum_{i=0}^{n} E_i d_{n_i}.$$

Note that $\sum_{i=0}^{n} d_{n_i} = 0$ since the number of balls does not change. Also, note that $dE = \sum_{i=0}^{n} E_i d_{n_i}$ is the change in system's energy. The above equation reduces to

$$d(\log W) = \lambda_1 \sum_{i=0}^{n} dn_i + \lambda_2 \sum_{i=0}^{n} E_i d_{n_i} = 0 + \lambda_2 \sum_{i=0}^{n} E_i d_{n_i} = \lambda_2 dE.$$

We now see that $\lambda_2 = \frac{d(\log W)}{dE}$, which we proved was constant at equilibrium (when we looked at the derivative of $\log W_A$ and $\log W_B$ with respect to dE_A). Therefore,

$$\lambda_2 = \frac{d(\log W)}{dE} = \frac{d(S/k_B)}{dE} = \frac{1}{k_B} \cdot \frac{dS}{dE} = \frac{1}{k_B T}.$$

Revisiting the Boltzmann distribution, we realize that we can write our previous result in the following way:

$$\rho(E_j) \propto e^{-\frac{E_j}{k_B T}}.$$

This is known as the *Boltzmann distribution*.

6 Einstein Stokes Relation

Going back to the physical analogy of the speck of dust in the water container, suppose that the only forces acting on the speck are gravity and drag force. Note that drag force is a function of velocity $F_{\text{drag}} = -\gamma v$, where γ is the drag coefficient of the water molecules and the negative sign indicates that the drag force opposes the direction of velocity. At equilibrium, the gravitational force is canceled out by the drag force:

$$F_g - F_{\text{drag}} = 0 \rightarrow F_{\text{drag}} = F_g.$$

Using the Newton's second law of motion ($F = ma$) and the formula for drag force, we get the following equation:

$$\gamma v = mg \rightarrow v = \frac{mg}{\gamma}.$$

The downward flux of particles due to drift is a product of the concentration and velocity. Let the concentration $\rho(z)$ be a function of the vertical position z, as the gravitational potential energy is determined by the altitude of the speck of dust. Then, the downward flux due to deterministic drift is

$$J_{\text{drift}} = -\rho(z)\frac{mg}{\gamma}.$$

We add the negative sign to to indicate that gravitational force points in the downward direction. At equilibrium, the upward flux due to random diffusion opposes and cancels out the downward flux due to drift. During diffusion, particles move from high density to low density regions. Diffusion flux is proportional to the negative of the concentration gradient:

$$J_{\text{diffusion}} = -D\frac{d\rho(z)}{dz}.$$

We then enforce the equilibrium condition:

$$J_{\text{drift}} + J_{\text{diffusion}} = 0$$
$$\rho(z)\frac{mg}{\gamma} - D\frac{d\rho(z)}{dz} = 0$$

From the Boltzmann distribution,

$$\rho(E_j) = \rho_0 e^{-\frac{E_j}{k_B T}} = \rho_0 e^{-\frac{mgz}{k_B T}}.$$

Note that we substitute $E_j = mgz$ since this is the physics formula for gravitational potential energy. Then, we take the derivative with respect to z:

$$\frac{d\rho(z)}{dz} = -\rho_0 \frac{mg}{k_B T} e^{-\frac{mgz}{k_B T}} = -\frac{mg}{k_B T}\rho(z).$$

Substituting back into the previous equilibrium condition, we get:

$$\begin{aligned}
-\rho(z)\frac{mg}{\gamma} - D\frac{d\rho(z)}{dz} &= 0 \\
-\rho(z)\frac{mg}{\gamma} - D\left(-\frac{mg}{k_B T}\rho(z)\right) &= 0 \\
-\rho(z)\frac{mg}{\gamma} + D\frac{mg}{k_B T}\rho(z) &= 0 \\
-\frac{mg}{\gamma} + D\frac{mg}{k_B T} &= 0 \\
-\frac{1}{\gamma} + D\frac{1}{k_B T} &= 0 \\
-D\frac{1}{k_B T} &= -\frac{1}{\gamma} \\
D &= \frac{k_B T}{\gamma}
\end{aligned}$$

Note that we can cancel $\rho(z)$ on both sides since it is guaranteed to be nonzero by the exponential function. This result tells us the exact value of the diffusion coefficient D, which is a function of temperature T and the drag force coefficient γ. This makes sense, since we would expect a higher diffusion coefficient with higher temperature T, since water molecules are moving more rapidly and colliding more frequently. Additionally, we would expect the diffusion coefficient to be lower with higher drag force coefficient γ since it is harder to move through honey (which has a higher drag coefficient) than water, so it is harder to diffuse in honey than water. This result is known as the *Einstein Stokes relation*.

7 Langevin Sampling

Now that we have done some exploration in energy states, we can relate this back to the process of diffusion for Machine Learning. We can think of the density $\rho(z)$ in the previous section as a proxy for the probability that our speck of dust is at the altitude z. Therefore, we can think of high energy states as low probability states and low energy states as high probability states. The process of diffusion in Machine Learning is trying to take our random image, which has low probability and high energy, to a real image, which has high probability and low energy. Here is the update rule for our image x:

$$x_{t+1} = x_t + \underbrace{\epsilon \cdot \nabla_x \log P(x_t)}_{\text{drift}} + \underbrace{\sqrt{2\epsilon} \cdot z}_{\text{diffusion}}.$$

This is similar to the update we saw in the introduction, but it has an additional diffusion term.

7.1 Drift Term

For the drift term, we go back to the Boltzmann distribution.

$$\rho(E_j) \propto e^{-\frac{E_j}{k_B T}}.$$

We can express the probability of each energy state as the right hand side multiplied by some normalization constant $\frac{1}{Z}$, where Z can be viewed as the integral of $e^{-\frac{E_j}{k_B T}}$. We divide by Z to ensure that the probabilities

sum to 1. We also use $P(x)$ to denote the probability of being in state x and $E(x)$ to denote the energy of state x:

$$P(x) = \frac{1}{Z} e^{-\frac{E(x)}{k_B T}}.$$

Taking the natural logarithm of both sides, we get

$$\log P(x) = \log \frac{1}{Z} - \frac{E(x)}{k_B T}.$$

Now, take the gradient with respect to x:

$$\nabla_x \log P(x) = 0 - \nabla_x \frac{E(x)}{k_B T} = -\frac{1}{k_B T} \nabla_x E(x).$$

We see the direct relationship between probability and energy. The gradient of the log probability is directly proportional to the negative gradient of the energy:

$$\nabla_x \log P(x) \propto -\nabla_x E(x).$$

So, minimizing the energy $E(x)$ is equivalent to maximizing the log probability $\log P(x)$. This makes it justified to optimize for maximizing $\log P(x)$.

7.2 Diffusion Term

The diffusion term simulates the random collision forces from the water molecules. The quantity z is a random variable sampled from the Gaussian normal distribution $N(0, 1)$. We use a normal distribution because the Central Limit Theorem tells us that the average of many independent random variables tends to follow a normal distribution.

The ϵ term is the same as the stepsize of the update at each timestep. The random noise is known as a *Brownian motion* process, and the variance of this process grows linearly with time. Therefore, the standard deviation of the noise grows with the square root of time. That is why the diffusion term has a $\sqrt{\epsilon}$ and not just ϵ. We will revisit the $\sqrt{2}$ term in the next section, but at a high level, this $\sqrt{2}$ factor allows for the deterministic drift and random diffusion to be balanced so that the system eventually settles into the most likely Boltzmann distribution (corresponding to the image with lowest energy and highest probability).

Putting the drift and diffusion terms together, we get the final update formula:

$$x_{t+1} = x_t + \epsilon \cdot \nabla_x \log P(x_t) + \sqrt{2\epsilon} \cdot z.$$

This is known as the *Langevin dynamics update rule* or the *Unadjusted Langevin Algorithm*. It is "unadjusted" because we have discretized the continuous SDE. The algorithm will converge, but with some error that is dependent on the stepsize ϵ.

8 Fluctuation-Dissipation Theorem

When the speck of dust moves, it has kinetic energy, which is expressed mathematically as $\frac{1}{2}mv^2$. Additionally, when the speck is in thermal equilibrium, its total energy is shared evenly between its 3 degrees of freedom (x, y, and z directions). Each degree of freedom has an average energy of $\frac{1}{2}k_B T$, so it has a total energy of $\frac{3}{2}k_B T$, but we will look at the one dimensional case for now (i.e. we will look at the energy and velocity in only one dimension, or one of the degrees of freedom). On average, the kinetic energy in this dimension is equal to $\frac{1}{2}k_B T$. This is a result of the *Equipartition Theorem* in physics. We will denote $\langle x \rangle$ as

the average value of x. Then, we can say:

$$\begin{aligned}\left\langle \frac{1}{2}mv^2 \right\rangle &= \frac{1}{2}k_BT \\ \frac{1}{2}m\left\langle v^2 \right\rangle &= \frac{1}{2}k_BT \\ \left\langle v^2 \right\rangle &= \frac{k_BT}{m}\end{aligned}$$

We are able to pull $\frac{1}{2}m$ out of the angle brackets since it is a constant and expectation is a linear function. Let us find a function for the velocity as a function of time. The key idea is using $F = ma$ and some calculus tricks:

$$\begin{aligned}ma &= F_{\text{total}} \\ m\frac{dv}{dt} &= F_{\text{drag}}(t) + F_{\text{random}}(t) \\ m\frac{dv}{dt} &= -\gamma v(t) + F_{\text{random}}(t) \\ \frac{dv}{dt} + \frac{1}{m}\gamma v(t) &= \frac{1}{m}F_{\text{random}}(t) \\ e^{\frac{\gamma}{m}t}\left(\frac{dv}{dt} + \frac{1}{m}\gamma v(t)\right) &= e^{\frac{\gamma}{m}t}\left(\frac{1}{m}F_{\text{random}}(t)\right) \\ \frac{d}{dt}\left(v(t)e^{\frac{\gamma}{m}t}\right) &= \frac{1}{m}e^{\frac{\gamma}{m}t}F_{\text{random}}(t) \qquad \text{(by the derivative product rule)} \\ \int_{-\infty}^{t}\left(\frac{d}{ds}\left(v(s)e^{\frac{\gamma}{m}s}\right)\right)ds &= \int_{-\infty}^{t}\left(\frac{1}{m}e^{\frac{\gamma}{m}s}F_{\text{random}}(s)\right)ds \\ \left[\frac{d}{ds}\left(v(s)e^{\frac{\gamma}{m}s}\right)\right]_{-\infty}^{t} &= \int_{-\infty}^{t}\left(\frac{1}{m}e^{\frac{\gamma}{m}s}F_{\text{random}}(s)\right)ds \\ v(t)e^{\frac{\gamma}{m}t} &= \frac{1}{m}\int_{-\infty}^{t}e^{\frac{\gamma}{m}s}F_{\text{random}}(s)\,ds \\ v(t) &= \frac{1}{m}\int_{-\infty}^{t}e^{-\frac{\gamma}{m}(t-s)}F_{\text{random}}(s)\,ds\end{aligned}$$

We take the integral from $-\infty$ to t since the velocity at time t is only affected by all time before t, not after t. In other words, this is a causal process. Another observation is that we can view the current velocity as a weighted average over all the past random forces, where the weight assigned to each force in at time s in the past is $e^{-\frac{\gamma}{m}(t-s)}$. The memory weight of each of the past random forces is an exponential decay function.

Now, note that F_{random}, or the random forces of the water molecules acting upon the speck of dust, are equivalent to the negative of the drag force in equilibrium:

$$\begin{aligned}F_{\text{drag}} + F_{\text{random}} &= 0 \\ F_{\text{random}} &= -F_{\text{drag}} = -(-\gamma v) = \gamma v\end{aligned}$$

Note that $F_{\text{random}}(t)$ is actually a function of time. Let us examine the variance of the random force $\text{Var}[F_{\text{random}}(t)]$. We do this by writing $\langle F_{\text{random}} \cdot v(t)\rangle$ in two different ways. First,

$$\begin{aligned}\langle F_{\text{random}} \cdot v(t)\rangle &= \langle \gamma v(t) \cdot v(t)\rangle \\ &= \gamma\left\langle v(t)^2\right\rangle \\ &= \frac{\gamma k_BT}{m}\end{aligned}$$

Second,

$$\begin{aligned}
\langle F_{\text{random}}(t) \cdot v(t) \rangle &= \left\langle F_{\text{random}}(t) \cdot \frac{1}{m} \int_{-\infty}^{t} e^{-\frac{\gamma}{m}(t-s)} F_{\text{random}}(s)\, ds \right\rangle \\
&= \frac{1}{m} \int_{-\infty}^{t} e^{-\frac{\gamma}{m}(t-s)} \left\langle F_{\text{random}}(t) \cdot F_{\text{random}}(s) \right\rangle \, ds \\
&= \frac{1}{m} \int_{-\infty}^{t} e^{-\frac{\gamma}{m}(t-s)} (\text{Var}[F_{\text{random}}(t)] \cdot \delta(t-s))\, ds \\
&= \frac{\text{Var}[F_{\text{random}}(t)]}{m} \frac{1}{2} e^{-\frac{\gamma}{m}(t-t)} \\
&= \frac{\text{Var}[F_{\text{random}}(t)]}{2m}
\end{aligned}$$

Here, we use the Dirac delta function to weight the products between the random forces at times t and s:

$$\delta(t-s) = \begin{cases} \infty & \text{if } t = s \\ 0 & \text{if } t \neq s \end{cases}$$

Note that we would like to put a weight of 0 if $t \neq s$ since the random forces are independent at different times. Also, note that the integral of the Dirac delta function is 1. Setting these two expressions equal to each other, we get:

$$\begin{aligned}
\frac{\gamma k_B T}{m} &= \frac{\text{Var}[F_{\text{random}}(t)]}{2m} \\
\text{Var}[F_{\text{random}}] &= 2\gamma k_B T
\end{aligned}$$

Going back to the forward and reverse SDE, we want the random diffusion term to look like

$$dx = \cdots + \underbrace{\sigma dW_t}_{\text{diffusion}},$$

where dW_t is the noise drawn from a normal distribution $N(0, dt)$. Therefore, we can write $dW_t = z\sqrt{dt}$, or $\Delta W_t = z\sqrt{\Delta t}$ in the discretized case. We would like to scale this noise by some σ that controls the strength of the random noise. To find out σ, we can calculate the value of σ^2 first, or the variance of the strength of the noise applied. Note that the square of the position of our speck of dust x^2 is the mean square displacement of the speck. Then, $\left\langle \frac{dx^2}{dt} \right\rangle = \frac{d\langle x^2 \rangle}{dt}$ is the rate of spreading. Note that

$$\frac{d(x^2)}{dt} = 2x\frac{dx}{dt} = 2xv.$$

At equilibrium, $F_{\text{random}}(t) = -F_{\text{drag}}(t) = \gamma v$. Therefore, $v = \frac{1}{\gamma} F_{\text{random}}(t)$. Then,

$$\begin{aligned}
\frac{d\langle x^2 \rangle}{dt} &= \langle 2xv \rangle \\
&= \left\langle 2x \frac{1}{\gamma} F_{\text{random}}(t) \right\rangle \\
&= \frac{2}{\gamma} \underbrace{\langle x \cdot F_{\text{random}}(t) \rangle}_{\text{Cov}(x, F_{\text{random}})}
\end{aligned}$$

Note that $\langle x \cdot F_{\text{random}}(t) \rangle = \text{Cov}(x, F_{\text{random}})$ since

$$\text{Cov}(x, F_{\text{random}}) = \langle x \cdot F_{\text{random}}(t) \rangle - \langle x \rangle \cdot \langle F_{\text{random}} \rangle = \langle x \cdot F_{\text{random}}(t) \rangle - 0 \cdot 0.$$

The average displacement and the average random force are both 0. Also, note that at equilibrium, $F_{\text{random}}(t) = -F_{\text{potential}}(t)$, so

$$\langle x \cdot F_{\text{random}}(t) \rangle = \text{Cov}(x, F_{\text{random}}) = -\text{Cov}(x, F_{\text{potential})} = -\langle x \cdot F_{\text{potential}}(t) \rangle ,$$

where $F_{\text{potential}}$ is the force due to potential energy. The force due to potential energy is equal to the negative derivative of potential energy:

$$F_{\text{potential}} = -\frac{dU(x)}{dx},$$

where $U(x)$ is the potential energy function. We get $\langle x \cdot F_{\text{potential}}(t) \rangle$ in the following way:

$$\begin{aligned}
\langle x \cdot F_{\text{potential}}(t) \rangle &= \int_{-\infty}^{\infty} (x \cdot F_{\text{potential}}(t)) \cdot P(x)\, dx \\
&= \int_{-\infty}^{\infty} x \left(-\frac{dU(x)}{dx} \right) \frac{1}{Z} e^{-\frac{U(x)}{k_B T}}\, dx \qquad \text{(from the Boltzmann distribution)}
\end{aligned}$$

Note that we use $U(x)$ in place of $E(x)$, as this is the potential energy function. Take the derivative of the exponential term and use the chain rule:

$$\frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right) = \left(-\frac{1}{k_B T} \frac{dU(x)}{dx} \right) e^{-\frac{U(x)}{k_B T}}.$$

Rearranging, we get:

$$\left(-\frac{dU(x)}{dx} \right) e^{-\frac{U(x)}{k_B T}} = k_B T \frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right).$$

Replacing this in the previous integral, we get:

$$\begin{aligned}
\langle x \cdot F_{\text{potential}}(t) \rangle &= \int_{-\infty}^{\infty} x \left(-\frac{dU(x)}{dx} \right) \frac{1}{Z} e^{-\frac{U(x)}{k_B T}}\, dx \\
&= \frac{1}{Z} \int_{-\infty}^{\infty} x k_B T \frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right) dx \\
&= \frac{k_B T}{Z} \int_{-\infty}^{\infty} x \frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right) dx
\end{aligned}$$

Now, we use integration by parts. Let

$$\begin{aligned}
u = x \to du = dx \\
dv = \frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right) dx \to v = e^{-\frac{U(x)}{k_B T}}
\end{aligned}$$

Then, we apply integration by parts:

$$\begin{aligned}
\int u\, dv = uv - \int v\, du \\
\frac{k_B T}{Z} \int_{-\infty}^{\infty} x \frac{d}{dx}\left(e^{-\frac{U(x)}{k_B T}} \right) dx = \frac{k_B T}{Z} \left[\left[x \cdot e^{-\frac{U(x)}{k_B T}} \right]_{-\infty}^{\infty} - \int_{-\infty}^{\infty} e^{-\frac{U(x)}{k_B T}}\, dx \right]
\end{aligned}$$

If we say that the particle has infinite potential energy ($U(x) = \infty$) at $x = \pm\infty$, then the entire term evaluates to 0. Additionally, $\int_{-\infty}^{\infty} e^{-\frac{U(x)}{k_B T}}\, dx = Z$ by the definition of Z, so we get:

$$\frac{k_B T}{Z}(0 - Z) = -k_B T.$$

Here is the final result:

$$\langle x \cdot F_{\text{random}}(t) \rangle = k_B T.$$

We know that the diffusion coefficient $D = \frac{k_B T}{\gamma}$ from the Einstein Stokes relation, which means that

$$\sigma^2 = \left\langle \frac{dx^2}{dx} \right\rangle = \frac{2}{\gamma} \langle x \cdot F_{\text{random}}(t) \rangle = \frac{2}{\gamma} k_B T = 2D.$$

Therefore, $\sigma = \sqrt{2D}$. Therefore,

$$\begin{aligned} dx &= \cdots + \sigma dW_t \\ &= \cdots + \sqrt{2D} \left(z\sqrt{dt} \right) \\ &= \cdots + \sqrt{2D \cdot dt} \cdot z \\ &= \cdots + \sqrt{2D\epsilon} \cdot z \end{aligned}$$

We replace dt with the discrete stepsize ϵ. When applying the Langevin dynamics update rule, we usually set $D = 1$ for simplicity, and the responsibility of the diffusion coefficient is absorbed into the constant ϵ. (This does not imply that the physical constant $D = 1$.)

9 Predictor-Corrector Sampling

Now, we can put everything together into two updates. First, the Predictor uses the reverse SDE

$$\Delta x = [f(x,t) - g(t)^2 s_\theta(x,t)]\Delta t + g(t)\sqrt{\Delta t} \cdot z,$$

to take one step back in time

$$x_{t-\Delta t} \approx x_t - \Delta x.$$

However, due to the discretization of the problem, there are slight inaccuracies in how we update x. To correct this, we use a Corrector that performs several steps of correction using the Langevin dynamics update rule:

$$x_{\text{new}} = x_{\text{old}} + \epsilon \cdot s_\theta(x_o ld, t) + \sqrt{2\epsilon} \cdot z.$$

Technically either method (Predictor or Corrector) should work on its own, but they tend to work very well in practice when combined. The Predictor can be thought of as the heavyweight update, while the Corrector provides small nudges to correct any inaccuracies due to discretization. When used together, the Corrector can be thought of as making the current image more plausible at the current noise level by maximizing the log probability. Note that both methods require only one neural network: $s_\theta(x,t)$. This is a function that is able to approximate $\nabla_x \log P(x_t)$.